Tutorial 4

> Tutorial 4: Pick and Place in MuJoCo with IK

Author: Debojit Das

1) Overview

This assignment implements a complete pick-and-place pipeline in MuJoCo using an off-the-shelf IK library first, then comparing with custom IK solutions. You will analyze workspace, design the table, randomize scenarios, collect demonstrations, and evaluate failures.

Successful Demonstrations

attachment:5b1e807c-e1c5-441c-a9dc-cf0f30bb9aaf:2025-10-10_17-29-51.mp4

attachment:14b8349d-f0a5-44c7-bdc2-afbe7ff3b5c2:2025-10-10_17-30-28.mp4

attachment:99154152-460e-4470-9d5f-8c54374800cc:2025-10-10_17-42-22.mp4

attachment:5b1e807c-e1c5-441c-a9dc-cf0f30bb9aaf:2025-10-10_17-29-51.mp4

attachment:14b8349d-f0a5-44c7-bdc2-afbe7ff3b5c2:2025-10-10_17-30-28.mp4

attachment:99154152-460e-4470-9d5f-8c54374800cc:2025-10-10_17-42-22.mp4

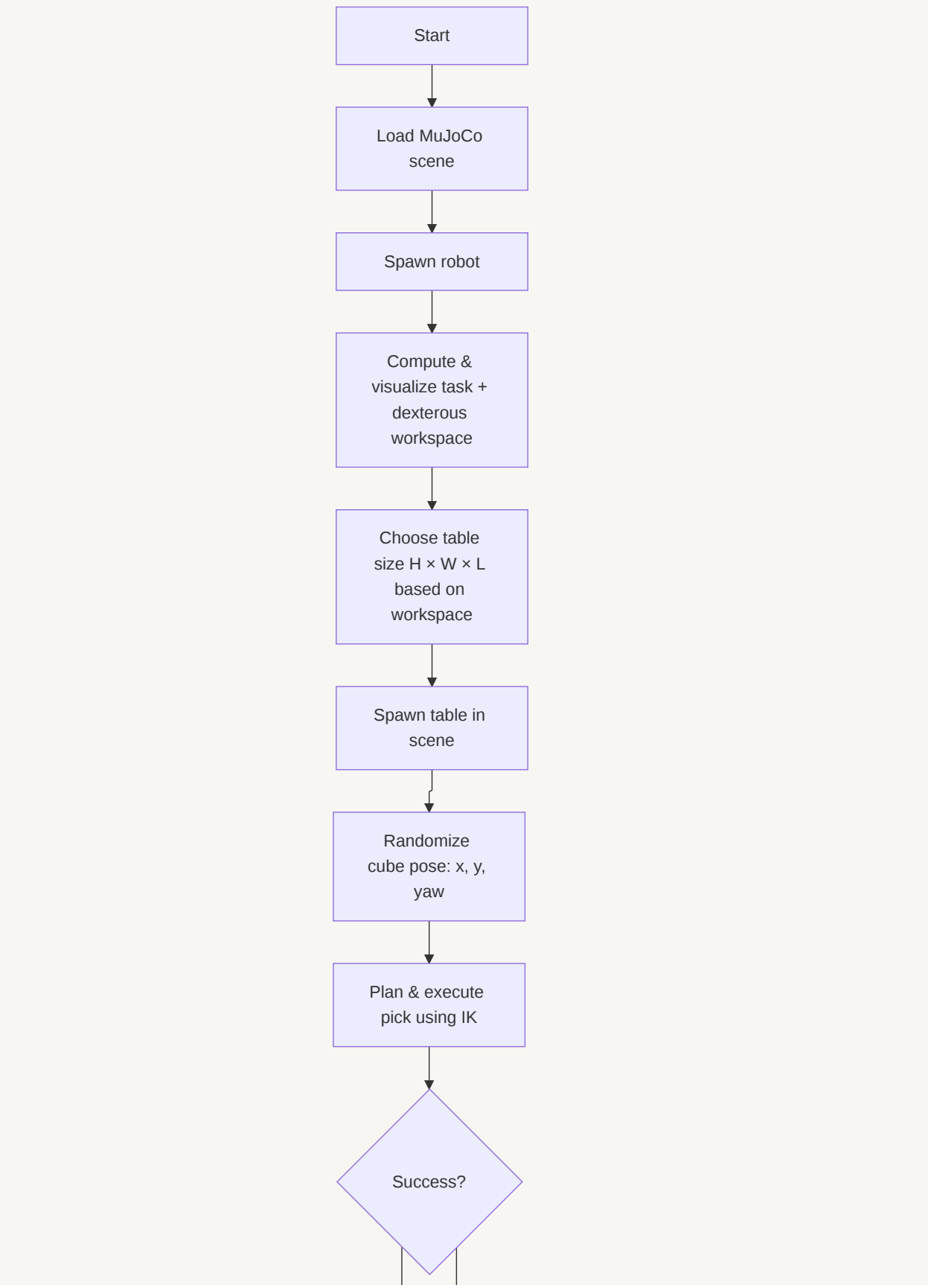
2) Learning goals

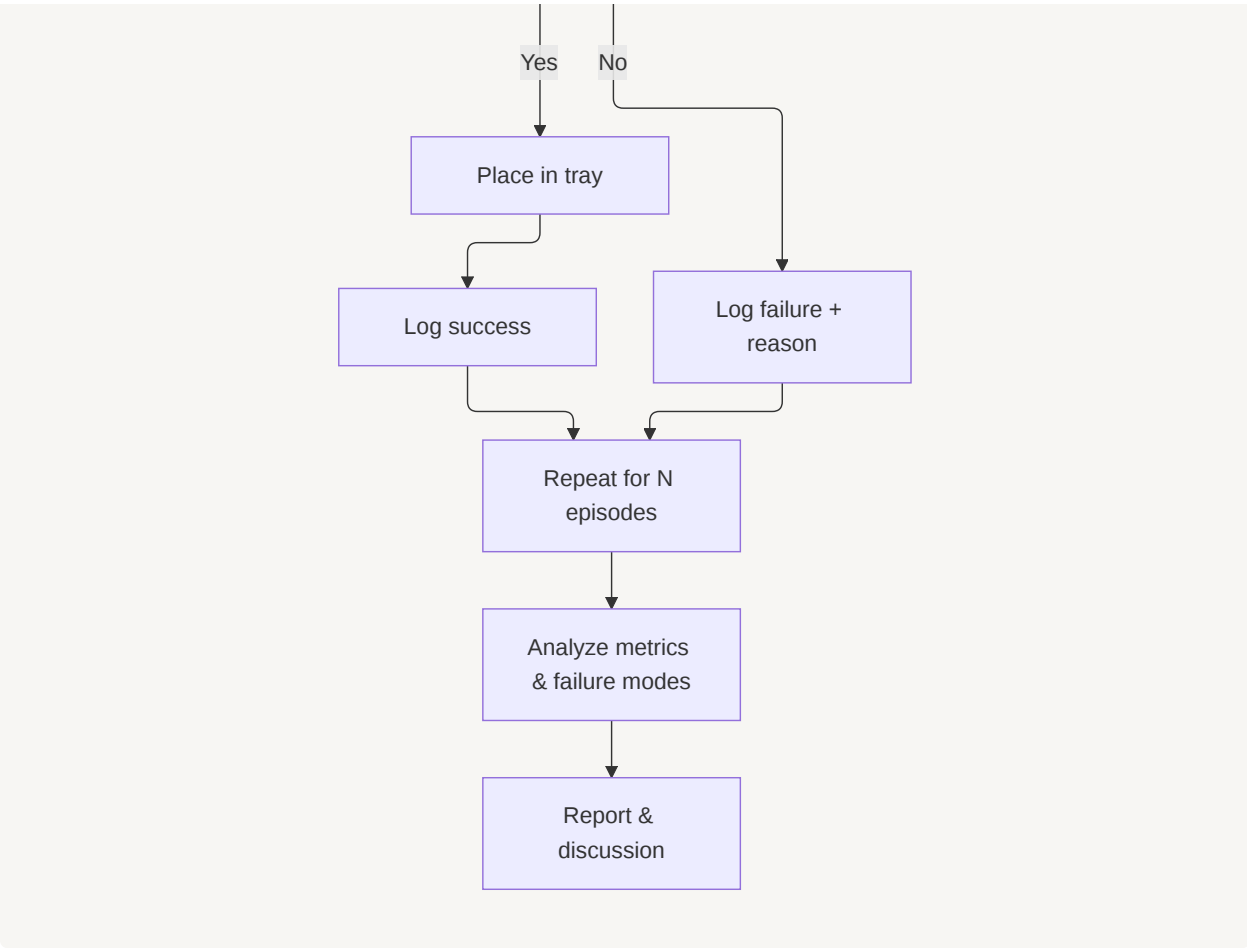
- Practice FK and extend to multiple IK approaches
- Analyze task and dexterous workspace and visualize them
- Design the workspace geometry for robust operation
- Implement randomized episodes and data collection at scale
- Evaluate failure cases and propose improvements

3) Prerequisites

- MuJoCo installed and working
- Python environment set up
- IK library: Mink (install and verify). Add link to the specific repo you choose in your report
- Basic plotting library for workspace visualization

4) Tasks and workflow


Start
Load MuJoCo scene
Spawn robot
Compute & visualize task + dexterous workspace
Choose table size H × W × L based on workspace
Spawn table in scene
Randomize cube pose: x, y, yaw
Plan & execute pick using IK
Success?



5) Workspace analysis

- Compute and visualize two workspaces:
  - Task workspace: reachable TCP positions for pick and place targets
  - Dexterous workspace: subset where arbitrary end-effector orientations about yaw are feasible
- Task workspace: reachable TCP positions for pick and place targets
- Dexterous workspace: subset where arbitrary end-effector orientations about yaw are feasible
- Plot both as point clouds or occupancy maps. Highlight feasible table regions

6) Table design

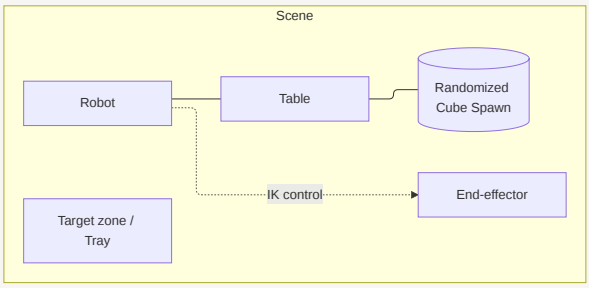
- Choose table dimensions based on the feasible area in your workspace plots

- Justify height to allow comfortable approach vectors and clearance

Parameter	Symbol	Value	Rationale
Table length	L	...	Fits within task workspace X-range
Table width	W	...	Fits within task workspace Y-range
Table height	H	...	Ensures vertical approach without joint limits

7) Environment setup

- Spawn the robot and the table in MuJoCo
- Load a cube model as the pick object
- Prepare a tray or target zone for placement



8) Episode generation and randomization

- For each episode, sample cube pose:

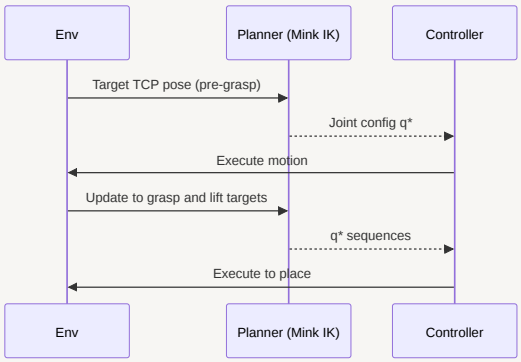
Variable	Range	Notes
x	[xmin, xmax]	Inside designed table surface
y	[ymin, ymax]	Inside designed table surface
yaw	[−π, π]	Uniform or task-specific

- Use a while loop to run N episodes and store logs per episode

9) IK Phase 1: Off-the-shelf IK (Mink)

- Implement grasp approach, grasp, lift, transit, and place using Mink

- Add basic collision checks between cube, gripper, table, and tray
- Log success or failure and reason



10) Batch demonstrations

- Run at least 25 episodes overnight
- Save per-episode logs: initial pose, success, failure reason, min distance margins, time to solve, IK iterations

Metric	Description
Success rate	Fraction of successful pick-and-place
Mean time to solve	Average planning time per episode
Min clearance	Minimum distance to obstacles
IK iterations	Iterations until convergence
Failure reason	E.g., IK infeasible, collision, grasp slip, limits

11) Failure analysis

- Categorize failures and attach representative screenshots or states
- Propose mitigations: better pre-grasp, different approach direction, alternate grasp yaw, collision margin tuning

Failure Cases Example

attachment:d983d026-ef90-4153-9270-cb6ff318dd03:2025-10-10_17-30-59.mp4

attachment:b385e035-6e8c-45c0-8355-168c2d9b3d94:2025-10-10_17-33-20.mp4

attachment:d983d026-ef90-4153-9270-cb6ff318dd03:2025-10-10_17-30-59.mp4

attachment:d983d026-ef90-4153-9270-cb6ff318dd03:2025-10-10_17-30-59.mp4

attachment:b385e035-6e8c-45c0-8355-168c2d9b3d94:2025-10-10_17-33-20.mp4

attachment:b385e035-6e8c-45c0-8355-168c2d9b3d94:2025-10-10_17-33-20.mp4

12) Stage 2: Custom IK methods

Replace the off-the-shelf IK with your implementations and compare:

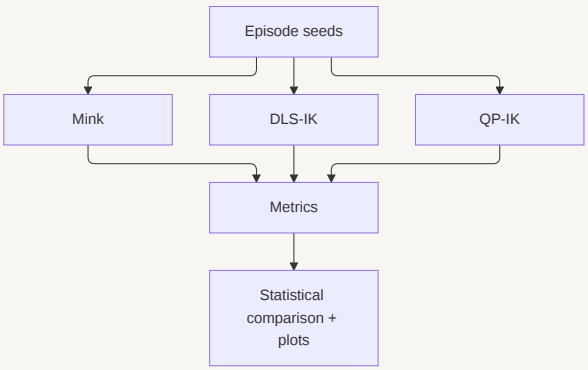
- Any valid IK method of your choice
- Damped Least Squares (closed-loop)
- Quadratic Programming with joint-limit constraints

Method	Strengths	Weaknesses	Notes
Off-the-shelf (Mink)	Fast to integrate	Less control over constraints	Baseline

Method	Strengths	Weaknesses	Notes
DLS-IK	Robust near singularities	Parameter tuning needed	Use damping λ
QP-based IK	Handles constraints cleanly	Requires solver setup	Add joint and velocity limits

13) Comparison protocol

- Use identical episode seeds for fair comparison
- Evaluate the same metrics as in Section 10
- Include joint limit and velocity constraint violations



14) Grading rubric

Item	Points
Successful demonstration of behavior	50
Smart collision avoidance between cube and tray	20
Workspace analysis and table design	10
Logging, plots, and discussion quality	10
Comparison of three IK methods	10
Total	100

15) Deliverables

- Code with clear README and instructions.

- Workspace plots and table design justification
- Logs and summary metrics table
- Failure case report with screenshots
- Comparative analysis of IK methods with plots
- **A 3-min video (1 min each IK method) showing different trails (Important)**



Please make your Repository link submissions on Google Classroom.
1) Code
2) Report
3) Video

16) Suggested report structure (Limit : 5 Pages)

1. Introduction and goals
2. Methods: environment, randomization, IK approaches
3. Results: metrics tables and plots
4. Failure analysis and mitigations
5. Discussion and conclusions

17) Tips

- Start with conservative approach poses and expand
- Log everything. Small details help explain failures
- Keep seeds fixed for comparisons
- Please understand your code and functions even if you are generating things using an AI agent since the code you generate will be used in further assignments and final projects.

Supplementary Content

Closed Form Damped Least Square Inverse Kinematics (DLS-IK)

We know this relation between end-effector velocities and joint velocities.

$$\dot{\mathbf{x}} = J(\mathbf{q})\,\dot{\mathbf{q}} \tag{1}$$

$$\dot{\mathbf{q}} = J(\mathbf{q})^{\dagger}\,\dot{\mathbf{x}}_d \tag{2}$$

$$J^{\dagger} = \begin{cases} J^{\top}(JJ^{\top})^{-1}, & \text{if } J \text{ is fat (underdetermined)} \\ (J^{\top}J)^{-1}J^{\top}, & \text{if } J \text{ is tall (overdetermined)} \end{cases} \tag{3}$$

$$\dot{\mathbf{q}} = (J^{\top}J)^{-1}J^{\top}\,\dot{\mathbf{x}}_d \tag{4}$$

Near singularities $J^{\top}J$ becomes ill-conditioned or singular. So we add a regularisation term λI with $\lambda > 0$.

$$\boxed{\dot{\mathbf{q}} = (J^{\top}J + \lambda I)^{-1}J^{\top}\dot{\mathbf{x}}_d} \tag{5}$$

Let us see if we can derive the same equation from a optimisation problem ...

We have two objectives what we wish to solve :

- Minimise the Task space Error
- Penalise large joint velocities

We have two objectives what we wish to solve :

- Minimise the Task space Error
- Penalise large joint velocities

- Minimise the Task space Error

- Penalise large joint velocities

We rewrite the IK as a regularised optimisation problem (Lavenberg - Marquardt Formulation)

$$\dot{\mathbf{q}} = \arg\min_{\dot{\mathbf{q}}} \left(\left\| J\dot{\mathbf{q}} - \dot{\mathbf{x}}_d \right\|^2 + \lambda \left\| \dot{\mathbf{q}} \right\|^2 \right) \tag{6}$$

$$\dot{\mathbf{q}} = \arg\min_{\dot{\mathbf{q}}} \left(\underbrace{\left\| J\dot{\mathbf{q}} - \dot{\mathbf{x}}_d \right\|^2}_{\text{tracking error}} + \underbrace{\lambda \left\| \dot{\mathbf{q}} \right\|^2}_{\text{regularisation (damping)}} \right) \tag{7}$$

Now, find the value of $\dot{q}$ that minimises $\mathcal{J}(\dot{\mathbf{q}})$,

$$\mathcal{J}(\dot{\mathbf{q}}) \;=\; \left\| J(\mathbf{q})\,\dot{\mathbf{q}} - \dot{\mathbf{x}}_d \right\|_2^2 \;+\; \lambda \left\| \dot{\mathbf{q}} \right\|_2^2, \qquad \lambda > 0. \tag{8}$$

$$\mathcal{J}(\dot{\mathbf{q}}) = (J\dot{\mathbf{q}} - \dot{\mathbf{x}}_d)^\top (J\dot{\mathbf{q}} - \dot{\mathbf{x}}_d) + \lambda\, \dot{\mathbf{q}}^\top \dot{\mathbf{q}} \tag{9}$$

$$\mathcal{J}(\dot{\mathbf{q}}) = \dot{\mathbf{q}}^\top J^\top J \dot{\mathbf{q}} - 2\dot{\mathbf{x}}_d^\top J \dot{\mathbf{q}} + \dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}_d + \lambda\, \dot{\mathbf{q}}^\top \dot{\mathbf{q}} \tag{10}$$

$$\mathcal{J}(\dot{\mathbf{q}}) = \dot{\mathbf{q}}^\top (J^\top J + \lambda I) \dot{\mathbf{q}} - 2\dot{\mathbf{x}}_d^\top J \dot{\mathbf{q}} + \dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}_d \tag{11}$$

$$\frac{\partial \mathcal{J}}{\partial \dot{\mathbf{q}}} = 2(J^\top J + \lambda I)\dot{\mathbf{q}} - 2J^\top \dot{\mathbf{x}}_d = 0 \tag{12}$$

$$(J^\top J + \lambda I)\dot{\mathbf{q}} = J^\top \dot{\mathbf{x}}_d \tag{13}$$

$$\boxed{\dot{\mathbf{q}} = (J^\top J + \lambda I)^{-1} J^\top \dot{\mathbf{x}}_d} \tag{14}$$

Let’s further do an analysis of the regularisation term,

Let’s apply SVD to the Jacobian

$$J = U\,\Sigma\,V^\top \tag{15}$$

$$J^\top J = V\,\Sigma^\top \Sigma\,V^\top \tag{16}$$

Now we add the regularisation term λI

$$J^\top J + \lambda I = V\,\Sigma^\top \Sigma\, V^\top + \lambda I \tag{17}$$

Since V is orthogonal, we can write V as,

$$\lambda I = V \lambda I V^\top \tag{18}$$

Hence,

$$J^\top J + \lambda I = V\left(\Sigma^\top \Sigma + \lambda I\right) V^\top \tag{19}$$

Taking the inversion,

$$\left(J^\top J + \lambda I\right)^{-1} = V\left(\Sigma^\top \Sigma + \lambda I\right)^{-1} V^\top \tag{20}$$

$$\left(\Sigma^\top \Sigma + \lambda I\right)^{-1} = \operatorname{diag}\left(\frac{1}{\sigma_1^2 + \lambda}, \frac{1}{\sigma_2^2 + \lambda}, \dots\right) \tag{21}$$

Hence stability by the regularisation term.

Optimisation Based DLS Inverse Kinematics Solver formulated as a Quadratic Programming Problem

$$\min_{\Delta \mathbf{q}} \quad \tfrac{1}{2}\Delta \mathbf{q}^\top H \Delta \mathbf{q} + c^\top \Delta \mathbf{q} \quad \text{s.t.} \quad G\Delta \mathbf{q} \le h \tag{22}$$

Let’s derive this,

$$\mathcal{J}(\dot{\mathbf{q}}) \;=\; \left\| J(\mathbf{q})\,\dot{\mathbf{q}} - \dot{\mathbf{x}}_d \right\|_2^2 \;+\; \lambda \left\| \dot{\mathbf{q}} \right\|_2^2, \qquad \lambda > 0. \tag{23}$$

$$\mathcal{J}(\dot{\mathbf{q}}) = (J\dot{\mathbf{q}} - \dot{\mathbf{x}}_d)^\top (J\dot{\mathbf{q}} - \dot{\mathbf{x}}_d) + \lambda\, \dot{\mathbf{q}}^\top \dot{\mathbf{q}} \tag{24}$$

$$\mathcal{J}(\dot{\mathbf{q}}) = \dot{\mathbf{q}}^\top J^\top J \dot{\mathbf{q}} - 2\dot{\mathbf{x}}_d^\top J \dot{\mathbf{q}} + \dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}_d + \lambda\, \dot{\mathbf{q}}^\top \dot{\mathbf{q}} \tag{25}$$

$$\mathcal{J}(\dot{\mathbf{q}}) = \dot{\mathbf{q}}^\top (J^\top J + \lambda I) \dot{\mathbf{q}} - 2\dot{\mathbf{x}}_d^\top J \dot{\mathbf{q}} + \dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}_d \tag{26}$$

$$\mathcal{J}(\Delta\mathbf{q}) = \frac{1}{\Delta t^2}\,\Delta\mathbf{q}^\top (J^\top J + \lambda I)\Delta\mathbf{q} \;-\; \frac{2}{\Delta t}\,\dot{\mathbf{x}}_d^\top J\,\Delta\mathbf{q} \;+\; \dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}_d \tag{27}$$

$$\mathcal{J}(\Delta\mathbf{q}) = \underbrace{\frac{1}{2}\Delta\mathbf{q}^\top \left(\frac{2}{\Delta t^2}(J^\top J + \lambda I)\right)\Delta\mathbf{q}}_{\text{Quadratic (Hessian) term}} + \underbrace{\left(-\frac{1}{\Delta t} J^\top \dot{\mathbf{x}}d\right)^\top \Delta\mathbf{q}}_{\text{Linear term}} + \underbrace{\dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}d}_{\text{Constant term}} \tag{28}$$

Now, we define :

$$H = \frac{2}{\Delta t^2}(J^\top J + \lambda I), \quad c = -\frac{1}{\Delta t} J^\top \dot{\mathbf{x}}_d \tag{29}$$

$$\min_{\Delta\mathbf{q}} \quad \frac{1}{2}\Delta\mathbf{q}^\top H \Delta\mathbf{q} + c^\top \Delta\mathbf{q} + \dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}_d \tag{30}$$

Since the constant term $\dot{\mathbf{x}}_d^\top \dot{\mathbf{x}}_d$ does not depend on the optimisation variable Δq, it does not affect the minimise. Hence we drop it, but it can also be retained when evaluating the final cost value.

$$\min_{\Delta\mathbf{q}} \quad \frac{1}{2}\Delta\mathbf{q}^\top H \Delta\mathbf{q} + c^\top \Delta\mathbf{q} \tag{31}$$

Now the subject to constraints, need to be identified separately, mostly inequality constraints.

For our case we will have the joint limits as some constraints,

$$\mathbf{q}_{\min} \le \mathbf{q} + \Delta\mathbf{q} \le \mathbf{q}_{\max} \tag{32}$$

$$\mathbf{q}_{\min} - \mathbf{q} \le \Delta\mathbf{q} \le \mathbf{q}_{\max} - \mathbf{q} \tag{33}$$

$$\text{We now rewrite the two inequalities:} \quad \begin{cases} \Delta\mathbf{q} \le \mathbf{q}_{\max} - \mathbf{q} \\ -\Delta\mathbf{q} \le -(\mathbf{q}_{\min} - \mathbf{q}) \end{cases} \tag{34}$$

$$\text{This can be written in matrix form as: } G\Delta\mathbf{q} \le h \tag{35}$$

$$\underbrace{\begin{bmatrix} \mathbb{I} \\ -\mathbb{I} \end{bmatrix}}_{G} \Delta\mathbf{q} \;\le\; \underbrace{\begin{bmatrix} \mathbf{q}_{\max} - \mathbf{q} \\ -(\mathbf{q}_{\min} - \mathbf{q}) \end{bmatrix}}_{h} \tag{36}$$

DLS - IK (Quaternion Computations)

Here we will derive how can we get angular velocities, from two orientation current orientation and target orientations but written in the form of quaternions.

Let

$$q_c = \begin{bmatrix} x_c & y_c & z_c & w_c \end{bmatrix}^\top, \qquad q_t = \begin{bmatrix} x_t & y_t & z_t & w_t \end{bmatrix}^\top$$

be the **current** and **target** unit quaternions.

Note, this is scalar-last convention where we have w at the end commonly used in ROS related frameworks, but some libraries like MuJoCo does scalar first convention so w apperars as the first element.

Then the **quaternion error**

$$q_e \;=\; q_t \;\otimes\; q_c^{-1},$$

where "$\otimes$" is quaternion multiplication.

If $q_{e,w} < 0$ (negative scalar part), flip the sign so that

$$q_e \leftarrow -\,q_e, \qquad \text{forcing } q_{e,w} \geq 0$$

("hemisphere continuity").

Write $q_e = \begin{bmatrix} \mathbf{v}_e^\top,\; w_e \end{bmatrix}^\top$ with $ve \in R3\mathbf{v}_e \in \mathbb{R}^3$.

The quaternion logarithm gives the **axis-angle vector**

$$\boxed{\boldsymbol{\varepsilon}_\omega \;=\; 2\,\mathrm{Log}(q_e) \;=\; \theta\,\mathbf{u} \;=\; \theta\,\frac{\mathbf{v}_e}{\sin\!\big(\theta/2\big)},}$$

with

$\theta \;=\; 2\arccos\!\big(\mathrm{clip}(w_e, -1, 1)\big)$

(the code collapses to zero if $\theta < 10^{-6}$).

Now from here we get angular velocities.

Secondary Functions in DLS - IK (optional)

$$\dot{\mathbf{q}} = (J^\top J + \lambda I)^{-1} J^\top \dot{\mathbf{x}}_d$$

In the same closed form solution, now we can also incorporate a null space objective,

$$\dot{\mathbf{q}} = (J^\top J + \lambda I)^{-1} J^\top \dot{\mathbf{x}}_d + \left(I - (J^\top J + \lambda I)^{-1} J^\top J\right) \dot{\mathbf{q}}_{\text{null}}$$

Now $\dot{\mathbf{q}}_{\text{null}}$ can be steered in two ways, one that it avoids joint limits,

$$\dot{\mathbf{q}}_{\text{null}} = \alpha \left(\frac{1}{2}(\mathbf{q}_{\text{min}} + \mathbf{q}_{\text{max}}) - \mathbf{q} \right)$$

Or we optimise for individual arm manipulability,

$$\dot{\mathbf{q}}_{\text{null}} = \beta \, \nabla_{\mathbf{q}} \left(\sqrt{\det(J(\mathbf{q}) J(\mathbf{q})^\top)} \right)$$

Reference Videos:

https://www.youtube.com/watch?v=nin2TbMuhR0&list=PLggLP4f-rq01fi62ek1BoV1yiPHLL3Vt-&index=1 : Watch these 3 to understand Inverse Kinematics

https://www.youtube.com/watch?v=6tj8QLF69Ok&list=PLggLP4f-rq00oA7lrv6dS5C15RYjqj4Zi : Jacobian

https://www.youtube.com/watch?v=hE_Duih_7JE&list=PLggLP4f-rq00efLcgMcG1m4k5CKlgRcGh : Watch these 3 for Forward Kinematics